

AWS re:Invent

NOV. 28 – DEC. 2, 2022 | LAS VEGAS, NV

ARC325

Well-Architected Reliability: Best practices you can use today

Seth Eliot (he/him)

Principal Developer Advocate,
AWS Developer Relations
AWS

Natalia Girolamo (she/her)

Senior Technical Program Manager,
AWS Well-Architected
AWS



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agenda

Well-Architected Reliability

- Definitions
- Best practices

Key takeaways and helpful resources

Answer *your* questions

Well-Architected Reliability



Definitions

Reliability

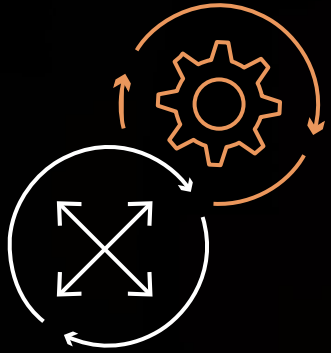
Ability of a workload to perform its required function correctly and consistently...

Resilience

Ability of a workload to recover from infrastructure or service disruptions...

– Reliability pillar, AWS Well-Architected Framework

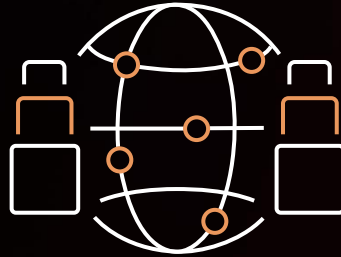
Six pillars of AWS Well-Architected Framework



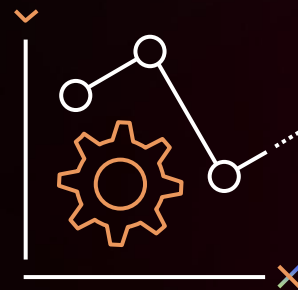
Operational
excellence



Security



Reliability



Performance
efficiency



Cost
optimization



Sustainability

What is your plan?



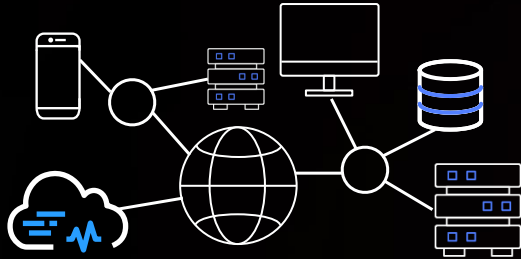
“We needed to build systems that embrace failure as a natural occurrence.”

Werner Vogels

CTO, Amazon.com



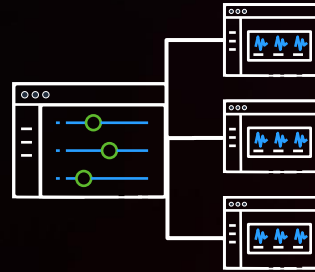
High reliability is now the new normal



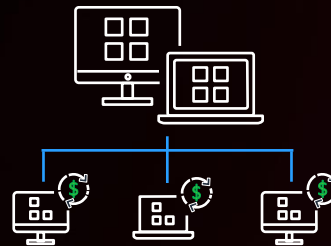
Complex ecosystems



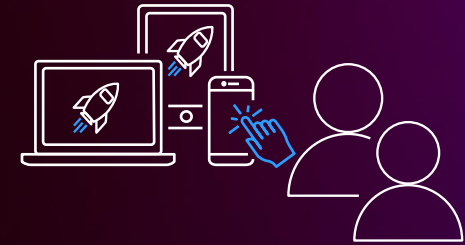
Multi-disciplinary
teams



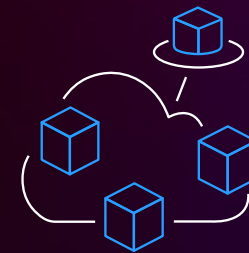
Incremental releases



Specialized services



Expanding channels



Always on

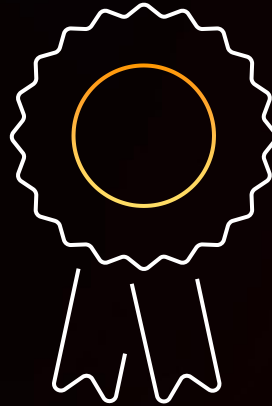
Well-Architected is a set of best practices



Design principles



Questions



Best Practices



Whitepaper



Labs

REL 12. How do you test reliability? [Info](#)

After you have designed your workload to be resilient to the stresses of production, testing is the only way to ensure that it will operate as designed, and deliver the resiliency you expect.

☐ Question does not apply to this workload [Info](#)

Select from the following

☒ Use playbooks to investigate failures [Info](#)

☒ Perform post-incident analysis [Info](#)

☒ Test functional requirements [Info](#)

☒ Test scaling and performance requirements [Info](#)

☒ Test resiliency using chaos engineering [Info](#)

☒ Conduct game days regularly [Info](#)

AWS Well-Architected Tool

AWS Well-Architected Tool

REL 12. How do you test reliability? [Info](#)

[Ask an expert](#) 

After you have designed your workload to be reliable, the only way to ensure that it will operate as designed is to test it.

☐ Question does not apply to this workload

Select from the following

- ☒ Use playbooks to investigate failures [Info](#)
- ☒ Perform post-incident analysis [Info](#)
- ☒ Test functional requirements [Info](#)
- ☒ Test scaling and performance requirements [Info](#)
- ☒ Test resiliency using chaos engineering [Info](#)
- ☒ Conduct game days regularly [Info](#)

REL 11. How do you design your workload to withstand component failures? [Info](#)

Workloads with a requirement for high availability and low mean time to recovery (MTTR) must be architected for resiliency.

☐ Question does not apply to this workload [Info](#)

Select from the following

- ☒ Monitor all components of the workload to detect failures [Info](#)
- ☒ Fail over to healthy resources [Info](#)
- ☒ Automate healing on all layers [Info](#)
- ☒ Use static stability to prevent bimodal behavior [Info](#)
- ☒ Send notifications when events impact availability [Info](#)

Best practices: Well-Architected Reliability

Four areas | 13 “questions”



Foundations



Workload architecture

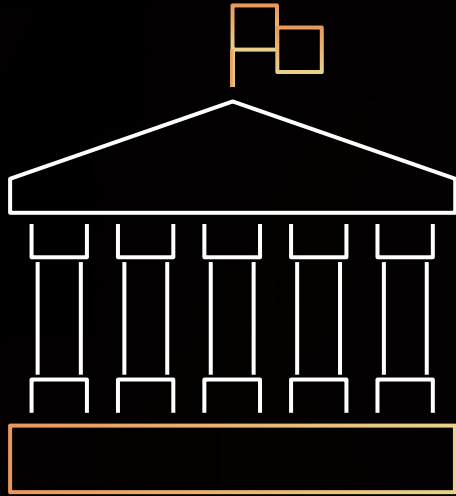


Change management



Failure management

Well-Architected Reliability pillar



Foundations

REL 1

Service quotas and limits

- ⌘ Know your AWS resource quotas
- ⌘ Raise quotas where necessary
- ⌘ Work within hard limits

REL 2

Network topology

- ⌘ Highly available endpoints
- ⌘ Redundant connections
- ⌘ Size and layout of network

Manage service quotas

Dashboard

**Amazon Athena**
Total quotas: 4

**Amazon Dynamo**
Total quotas: 9

**Amazon Elastic Compute Cloud (Amazon EC2)**
Total quotas: 103

**Amazon Relational Service (Amazon**
Total quotas: 27

**AWS CloudFormation**
Total quotas: 24

**AWS Key Manage (AWS KMS)**
Total quotas: 52

Pending service quota requests (0)

< 1 >

Quota name	Status	Last updated date
------------	--------	-------------------

Recently resolved service quota requests (0)

< 1 >

Quota name	Status	Last updated date
------------	--------	-------------------

Service Quotas > AWS services > AWS Lambda

AWS Lambda

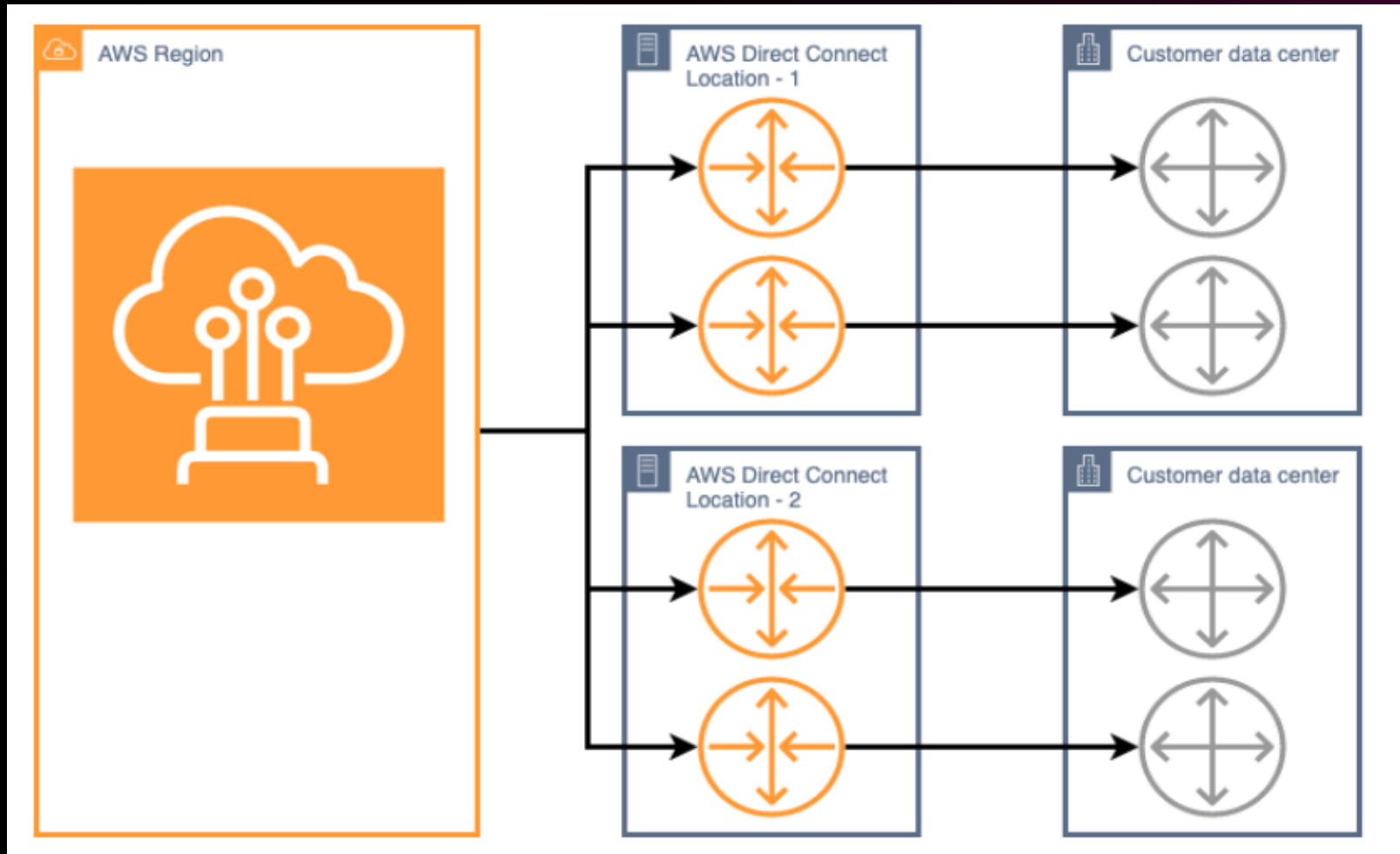
Service quotas

Request quota increase

< 1 > ⚙

Quota name	Applied quota value	AWS default quota value	Adjustable
Asynchronous payload	Not available	256 kilobytes	No
Burst concurrency	Not available	1,000	No
Concurrent executions	1,000	1,000	Yes

Provision redundant connectivity



Well-Architected Reliability pillar



Workload architecture

REL 3

Service architecture

- ⌘ Business domains
- ⌘ Right-sized, decoupled services

REL 4

Prevent failures

- ⌘ Loosely coupled dependencies
- ⌘ Idempotent, consistent

REL 5

Mitigate failures

- ⌘ Graceful degradation
- ⌘ Throttle, fail fast, limit retries

Choose how to segment your workload



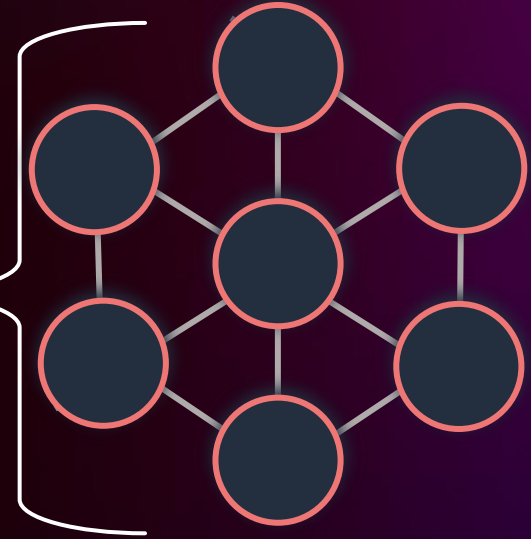
Monolithic application

- Does everything
- Shared release pipeline
- Rigid scaling
- High impact of change
- Hard to adopt new technologies



Service oriented

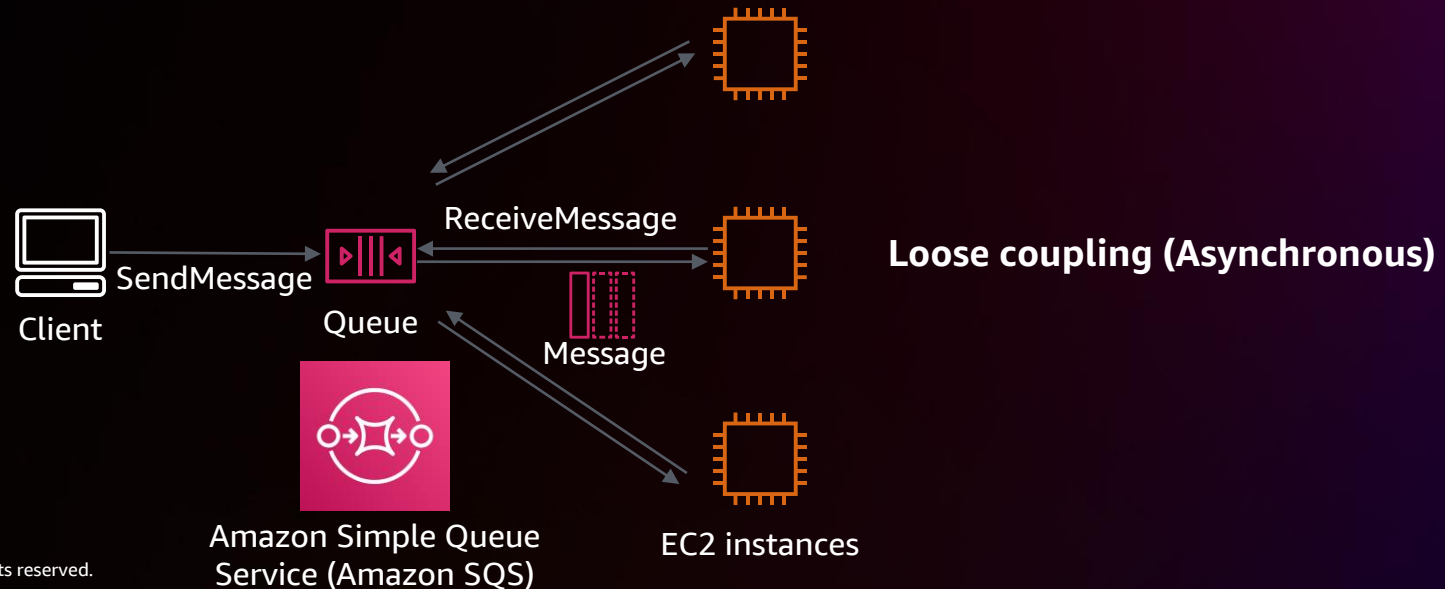
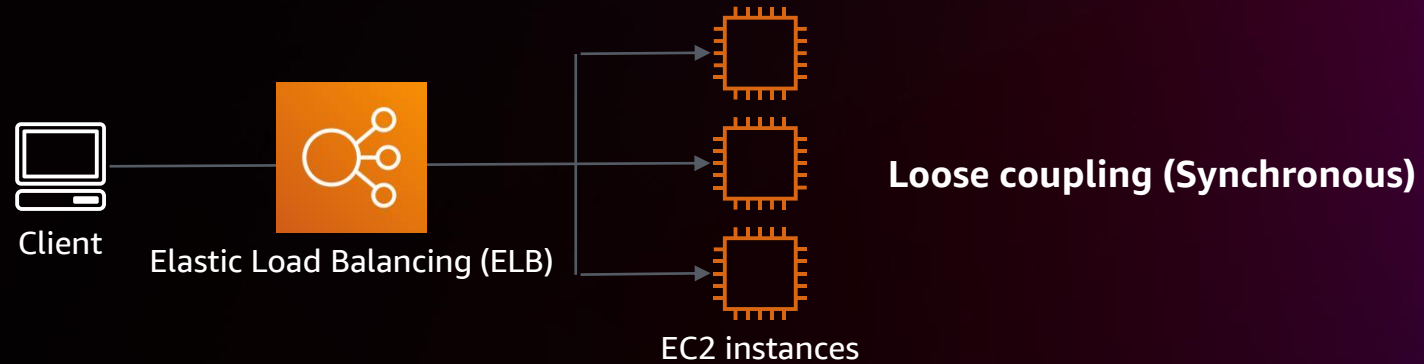
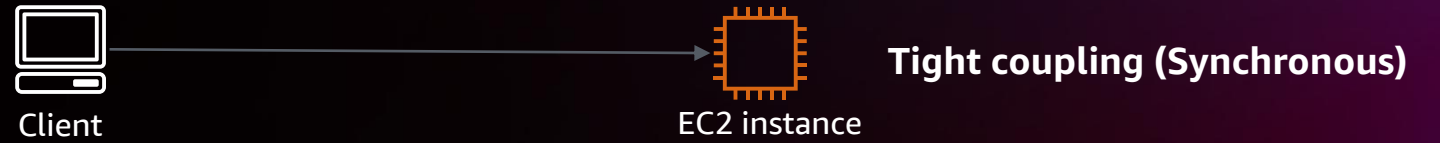
- Does some things
- Services surfaced via comms protocol
- Some coupled services



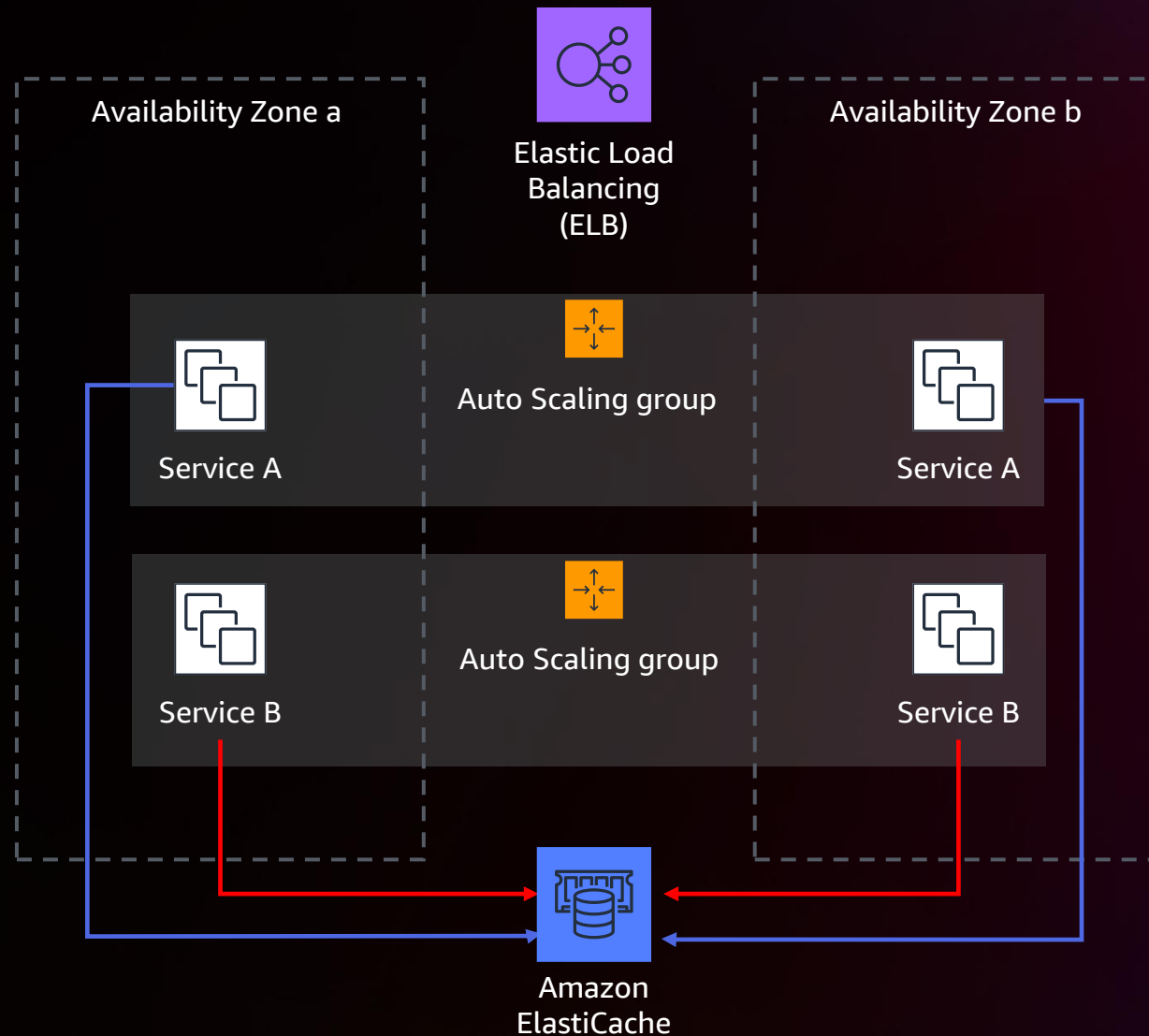
Microservices

- Does one thing
- Independent deployments
- Independent scaling
- Small impact of change
- Choice of technology

Implement loosely coupled dependencies



Make services stateless where possible



Well-Architected Reliability pillar



REL 6

Monitor workload resources

- ⌘ You can't manage what you can't see
- ⌘ Implement observability

REL 7

Adapt to changes in demand

- ⌘ Scale up and down as needed
- ⌘ Use automation

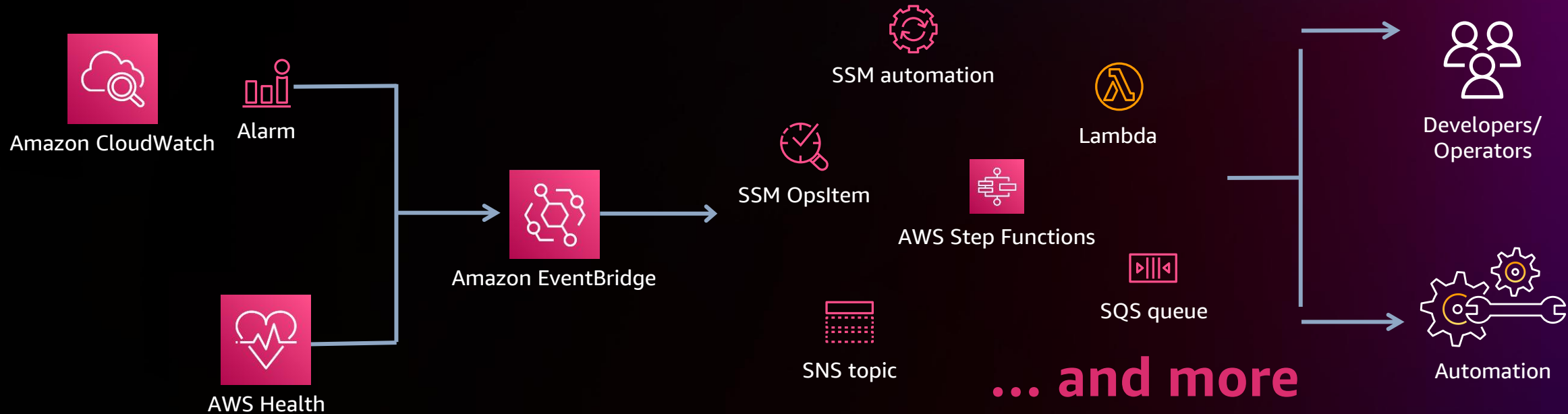
REL 8

Implement change

- ⌘ Deployments can be dangerous
- ⌘ Manage and reduce negative impacts

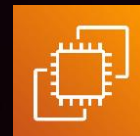
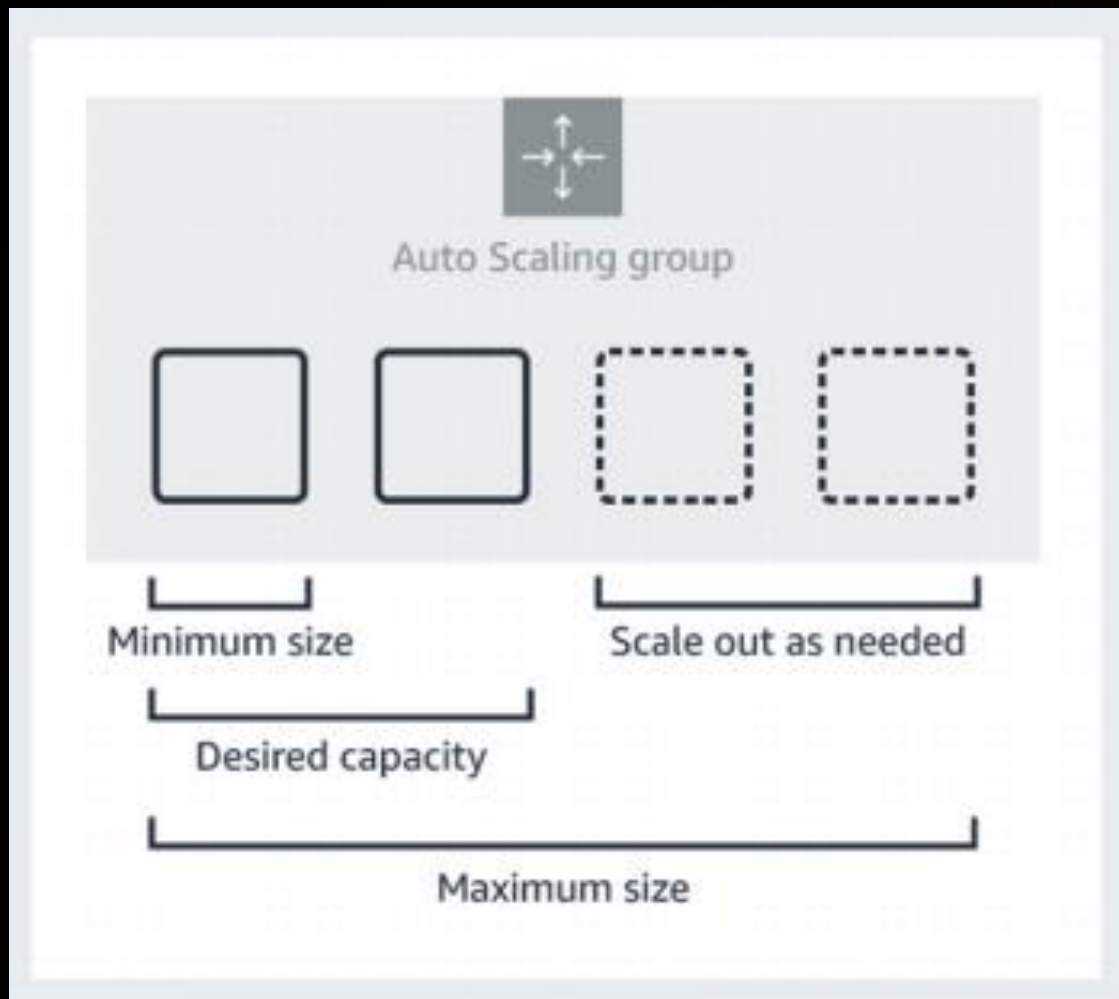
Real-time processing and alarming

- Send notifications
- Automate responses



```
{
  "source": ["aws.health"],
  "detail-type": ["AWS Health Event"],
  "detail": {
    "service": ["S3"],
    "eventTypeCategory": ["issue"],
    "eventTypeCode": ["AWS_S3_INCREASED_GET_API_ERROR_RATES",
"AWS_S3_INCREASED_PUT_API_ERROR_RATES"]
  }
}
```

Use automation when obtaining or scaling resources



Amazon EC2 instances



Amazon Aurora read-replicas



Amazon Aurora storage



Amazon DynamoDB throughput



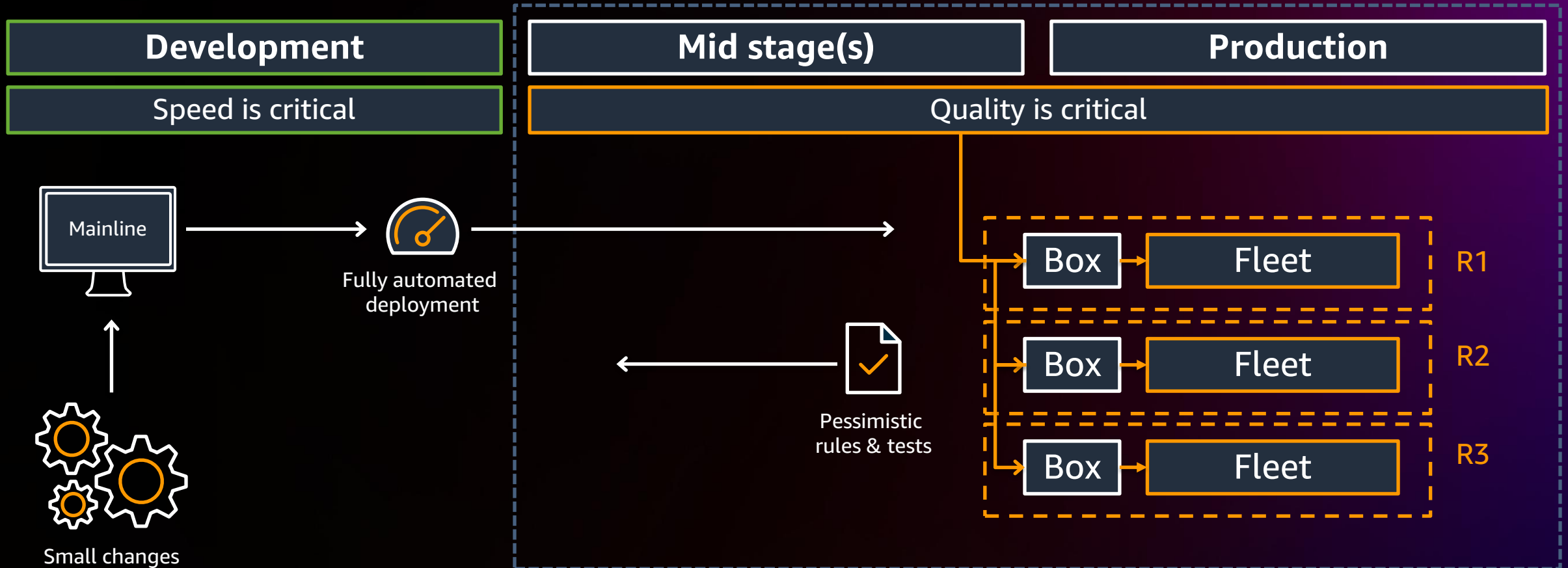
Amazon ECS tasks



AWS Lambda concurrent instances

Deploy changes with automation

Also... Integrate testing as part of your deployment



Well-Architected Reliability pillar



Failure management

REL 9

Back up data

- ⌘ Identify data to back up
- ⌘ Automate and secure backup

REL 10

Fault isolation

- ⌘ Use multiple sites
- ⌘ AWS Region, Availability Zone, etc.

REL 11

Withstand component failures

- ⌘ Automate recovery
- ⌘ Notify when failures occur

Deploy the workload to multiple locations

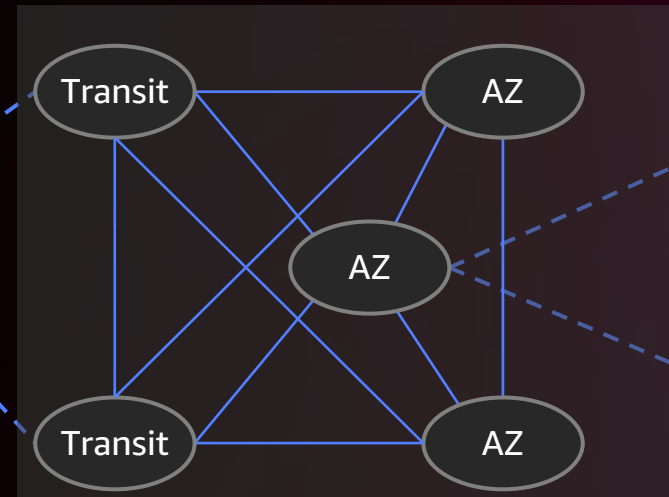
AWS REGIONS AND AVAILABILITY ZONES (AZS)

30 AWS Regions worldwide



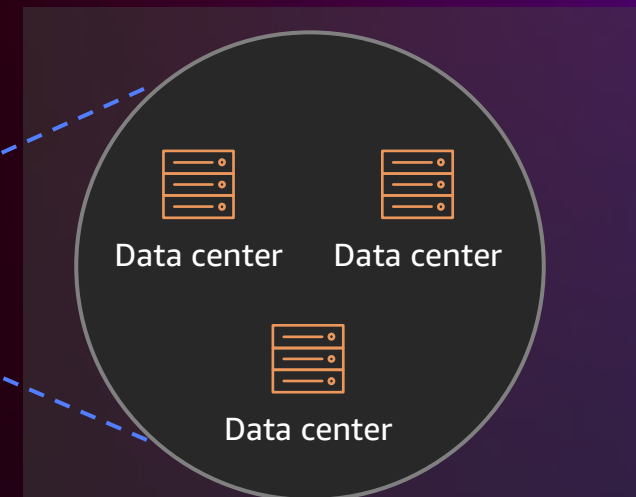
- AWS Regions
- Announced Regions

Each AWS Region has multiple AZs



A Region is a physical location in the world

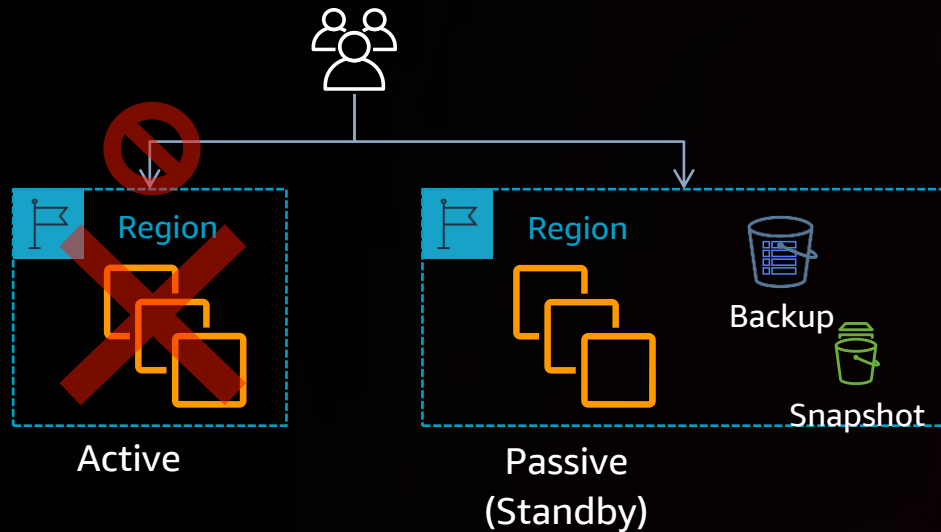
Each AZ includes one or more discrete data centers



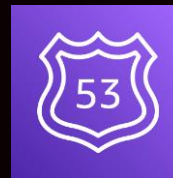
Data centers, each with redundant power, networking, and connectivity, housed in separate facilities.

Rely on the data plane, not the control plane

DURING RECOVERY

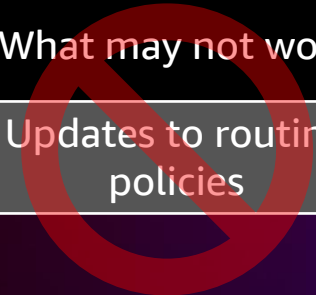


If control plane fails....



Amazon
Route 53

What will work	What may not work
DNS resolution and health checks	Updates to routing policies



Route 53
Application Recovery
Controller

Changes to routing controls	CRUDL routing controls
-----------------------------	------------------------



Well-Architected Reliability pillar



Failure management

REL 12 Test reliability

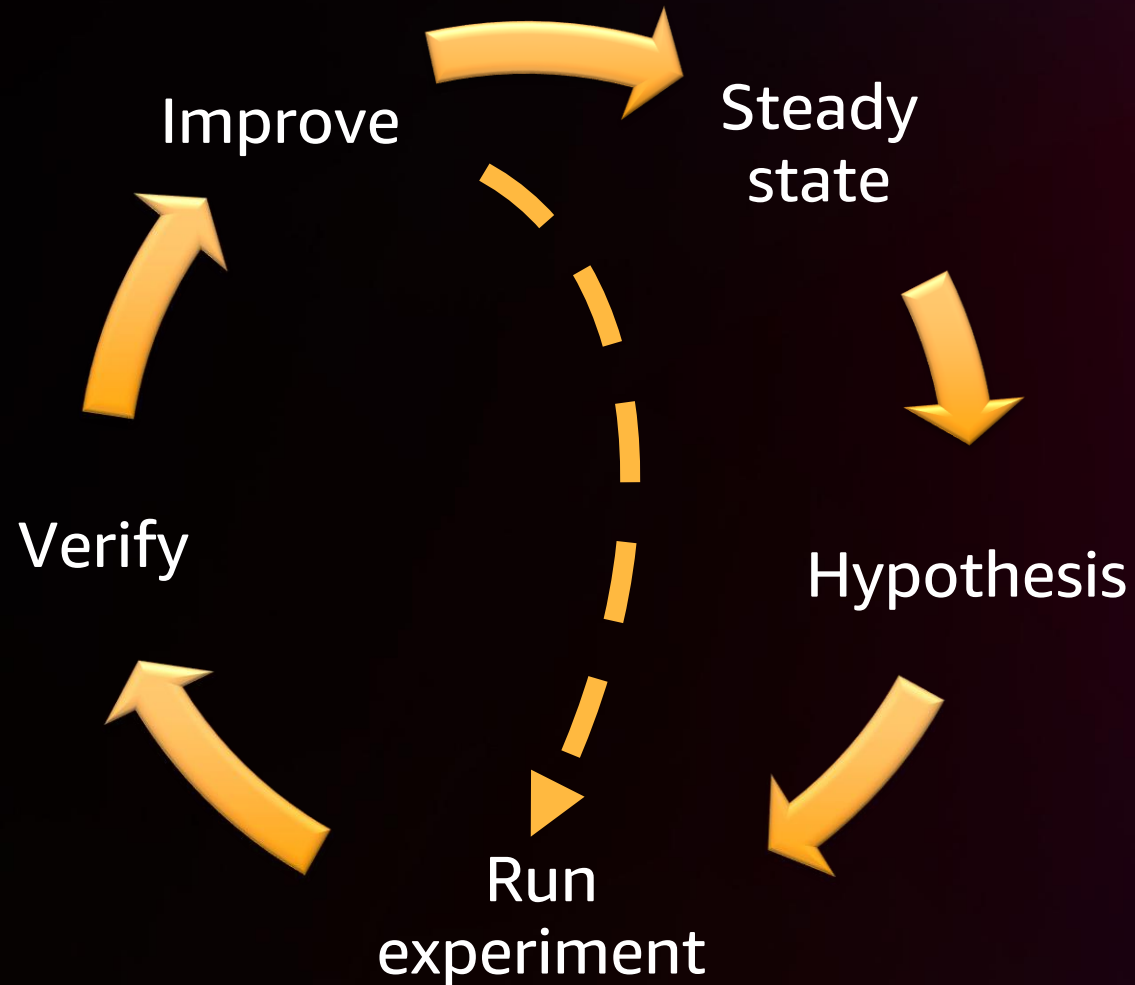
- ⌘ Simulate failure modes
- ⌘ Test scaling, performance, processes

REL 13 Disaster recovery (DR)

- ⌘ Set objectives
- ⌘ Implement and test DR strategy

Test resilience using chaos engineering

A SCIENTIFIC METHOD



Conduct game days

SIMULATE FAILURE OR EVENT TO TEST SYSTEMS' RESILIENCY, PROCESSES, AND TEAM RESPONSES



Planning

Hypothesis



Execution

Run experiment



Analysis

Verify
Improve

Conduct game days

SIMULATE FAILURE OR EVENT TO TEST SYSTEMS' RESILIENCY, PROCESSES, AND TEAM RESPONSES



People

Cross-discipline team
Processes



Briefing

Overview
Roles



Planning

Preparation
Hypothesis



Execution

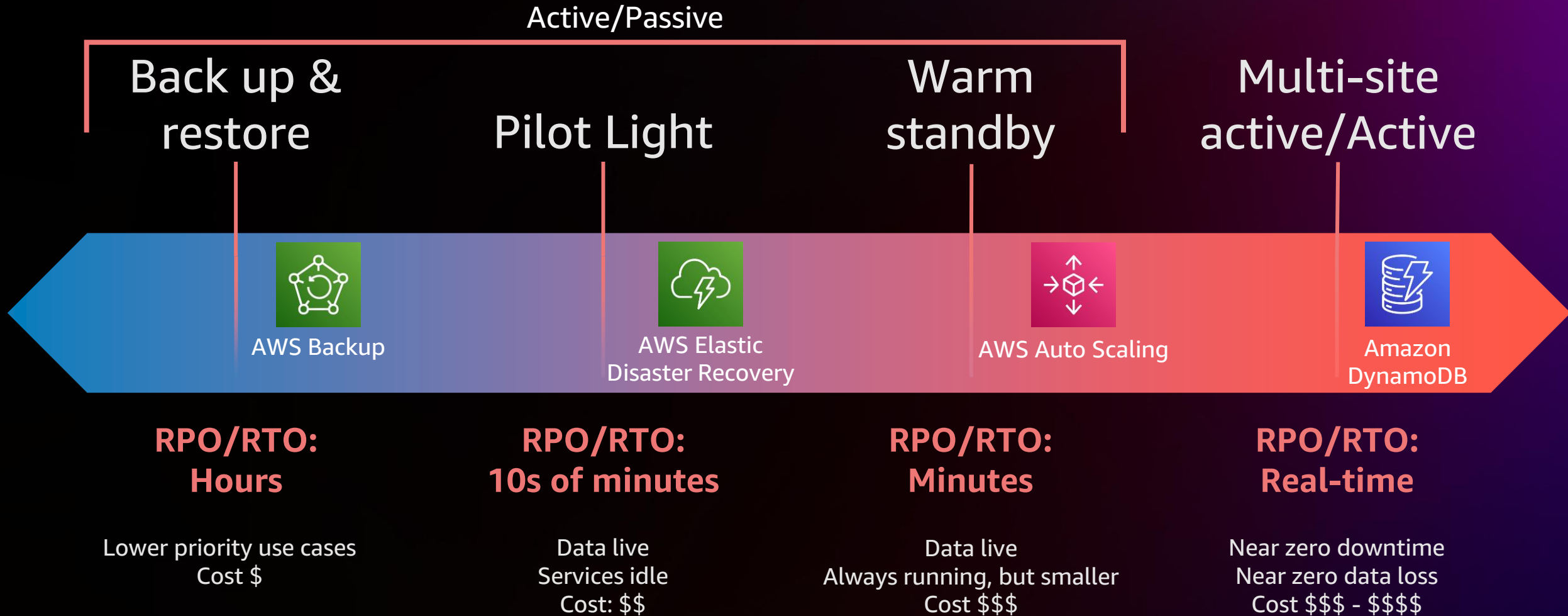
Run experiment



Analysis

Verify
Improve

Use defined recovery strategies for DR



Key takeaways and helpful resources

Dr PET

DRIVING RESILIENCE WITH PROACTIVE ENGAGEMENT BY TAM

Guidance from AWS subject matter experts to improve your Playbooks and disaster recovery process.

Dr PET Engagement Stages



Have questions or want to set up a review for your application?
Contact your Technical Account Manager (TAM)
(for customers with Enterprise Support)

Key takeaways

- ⌘ Reliability is a continuous improvement process
- ⌘ Your system is not reliable and resilient unless you test that
- ⌘ Prepare for failure – Evaluate risks
- ⌘ Call to action – Get started with the best practices mentioned!!!

Resources

Upcoming sessions

ARC208 – Workshop – Prepare & protect your applications from disruption with AWS Resilience Hub

ARC304 – Workshop – Build for resilience using Amazon Route 53 Application Recovery Controller

PRT268 – Breakout session – Camada Zero: A real-world architecture framework (sponsored by Itaú-Unibanco)

Whitepaper – Reliability pillar: AWS Well-Architected Framework

bit.ly/reliability-pillar

Well-Architected hands-on labs

wellarchitectedlabs.com/reliability

Well-Architected Tool

aws.com/well-architected-tool/



Thank you!

Seth Eliot

 [linkedin.com/in/setheliot](https://www.linkedin.com/in/setheliot)

 @setheliot

Natalia Girolamo

 [linkedin.com/in/nataliagirolamo](https://www.linkedin.com/in/nataliagirolamo)



Please complete the session survey in the **mobile app**



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.